Go Learning Journey

Phase 3 — Backend Development

■ 4–6 Weeks	■ 3 Projects	■ 3 Stages	■ Postgres + JWT
Duration	Hands-on	Covered	From Stage 2

Prerequisites: Phase 1 & Phase 2 complete — goroutines, channels, sync primitives, context, net/http client, encoding/json, os/io/bufio, and the Phase 2 net/http server (message-passing API).

This phase covers what turns a Go program into a production backend service: hand-rolled HTTP routing and middleware, a real relational database via `database/sql` and PostgreSQL, and authentication with JWTs — all built on the standard library first, before any framework enters the picture.

Curriculum source: `getsetgo.pdf`, pages 10–11. Sequencing below is reordered from the raw PDF bullets into an actual build order.

Table of Contents

01 **Mental Model First**
Why stdlib before Chi, the JSON envelope contract, status-code discipline

02 **Stage 1 — stdlib HTTP Fundamentals**
ServeMux/PathValue, middleware chains, error envelopes, graceful shutdown

03 **Stage 2 — Database Layer**
database/sql, sqlx, pgx, goose migrations, CRUD, aggregation

04 **Stage 3 — Auth**
bcrypt, JWT sign/verify, access/refresh lifecycle, revocation

05 **Drills**
12 progressive exercises with goals and gotchas

06 **Project 1 — REST API, No Framework**
Feature-based build: routing, middleware, envelopes, shutdown, tests

07 **Project 2 — URL Shortener + PostgreSQL**
Migrations, CRUD, redirects, click analytics

08 **Project 3 — Auth Service**
Signup/login, JWT issuance, refresh rotation, blocklist revocation

09 **Patterns Reference**
Middleware chaining, repository pattern, context-aware queries, claims design

10 **Common Mistakes & Gotchas**
SQL injection, N+1s, JWT pitfalls, shutdown races

11 **Resources & What's Next**
Canonical reads, Phase 4 preview

01 Mental Model First

Before writing a single handler, get these three ideas locked in — they shape every decision across all three stages of this phase.

Why stdlib before Chi (or Gin, Echo, Fiber)

Every popular Go web framework is a thin layer over `net/http`. Chi's router still returns an `http.Handler`. Its middleware is still `func(http.Handler) http.Handler`. If you learn a framework first, you learn its abstractions without knowing what they're abstracting — and you can't debug what you don't understand underneath. Build the pain manually once (path params without a router, middleware without a chain helper, JSON errors without a library), then Chi will read as "the same thing, less boilerplate" instead of magic.

The JSON Envelope Contract

In JS/TS backends you may have returned raw objects or ad-hoc shapes per route. In this phase, every single response — success or error — has one predictable shape. This is a discipline, not a stdlib feature:

```
{ "data": { ... }, "error": null }
{ "data": null, "error": { "code": "not_found", "message": "..." } }
```

A client (or your future self debugging with curl) never has to guess the shape of a response before parsing it.

Status Code Discipline

Status	When to use it
200 OK	Successful GET, PUT, or PATCH with a body returned
201 Created	Successful POST that created a resource — include Location header if practical
204 No Content	Successful DELETE, or PUT/PATCH with nothing to return
400 Bad Request	Malformed JSON, failed validation — client's fault, fixable by changing the request
401 Unauthorized	Missing or invalid credentials — "who are you?"
403 Forbidden	Valid credentials, insufficient permission — "I know who you are, you can't do this"
404 Not Found	Resource genuinely does not exist
409 Conflict	State conflict — e.g. duplicate unique key on insert
500 Internal Server Error	Your bug. Never leak the raw Go error string to the client.

■ The JS/TS instinct to return 200 with an `{ok: false}` body and let the frontend branch on it does not carry over. Go backends (and REST conventions generally) put the signal in the status code first, body second.

02 Stage 1 — stdlib HTTP Fundamentals

Built feature-by-feature, not as one monolithic rewrite of the Phase 2 server — you're adding new patterns on top of what you already know, so each feature below is its own small unit of work you can drop into any handler you build next.

ServeMux + PathValue Routing (Go 1.22+)

You already used this in the Phase 2 server. The next level is method + wildcard patterns on the same path, and knowing when ServeMux's built-in matching is enough vs. when you'd reach for a router.

main.go

```
mux := http.NewServeMux()
mux.HandleFunc("GET /items/{id}", getItem)
mux.HandleFunc("POST /items", createItem)
mux.HandleFunc("DELETE /items/{id}", deleteItem)

// inside getItem:
id := r.PathValue("id")
```

■ ServeMux does exact-vs-wildcard precedence automatically (more specific pattern wins), but it has no route groups and no built-in middleware — that's the gap Chi fills later.

Handler vs HandlerFunc

`http.HandlerFunc` is a type adapter: it lets a plain function satisfy the `http.Handler` interface. Understanding this unlocks why middleware signatures look the way they do.

```
type Handler interface {
    ServeHTTP(w http.ResponseWriter, r *http.Request)
}

type HandlerFunc func(http.ResponseWriter, *http.Request)

func (f HandlerFunc) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    f(w, r)
}
```

Request Parsing & Validation

decode + validate

```
func decode[T any](r *http.Request) (T, error) {
    var v T
    if err := json.NewDecoder(r.Body).Decode(&v); err != nil {
        return v, fmt.Errorf("decode: %w", err)
    }
    return v, nil
}
// validation is a separate step — keep decode errors (400, malformed)
// distinct from validation errors (422 or 400, semantically wrong)
```

Response Envelope Helper

respond.go

```
func writeJSON(w http.ResponseWriter, status int, data any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(Envelope{Data: data})
}

func writeError(w http.ResponseWriter, status int, code, msg string) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(Envelope{Error: &APIError{Code: code, Message: msg}})
}
```

Middleware Chain Pattern

The JS/TS equivalent is Express middleware (`(req, res, next) => {}`). Go's version wraps a handler in a function that returns a handler — no `next()` call, just calling the wrapped handler directly.

```
type Middleware func(http.Handler) http.Handler

func Logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
    })
}

func Chain(h http.Handler, mws ...Middleware) http.Handler {
    for i := len(mws) - 1; i >= 0; i-- {
        h = mws[i](h)
    }
    return h
}
// usage: Chain(mux, Logging, Recovery, Auth)
```

Graceful Shutdown via Context

main.go

```
srv := &http.Server{Addr: ":8080", Handler: mux}

go func() {
    if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
        log.Fatal(err)
    }
}()
```

```
stop := make(chan os.Signal, 1)
signal.Notify(stop, os.Interrupt, syscall.SIGTERM)
<-stop

ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
srv.Shutdown(ctx) // waits for in-flight requests, then exits
```

■ srv.Shutdown blocks new connections immediately but lets in-flight requests finish — this is the correct way to deploy Go services without dropping requests mid-request during a rolling restart.

03 Stage 2 — Database Layer

Every function below the handler layer that touches the DB should accept `ctx context.Context` as its first parameter — same contract you already follow for HTTP calls in Phase 2.

database/sql Core Types

Type	What it is
sql.DB	Not a single connection — a pool. Safe for concurrent use across goroutines. Open once, reuse everywhere.
sql.Rows	Streaming result set. Must call rows.Close() — defer it immediately, even on error paths.
sql.Row	Result of QueryRowContext — Scan() surfaces sql.ErrNoRows if nothing matched.
sql.Stmt	A prepared statement — reusable across multiple executions, one round-trip to parse.
sql.Tx	A transaction — Commit() or Rollback(), always via defer with a checked commit flag.

pgx — Driver + Toolkit

pgx is Postgres-specific (unlike lib/pq, which is a generic database/sql driver). Use it via `pgxpool` for the connection pool, and it also works as a plain database/sql driver if you register it that way — but going native pgx gets you better type mapping and performance.

db.go

```
pool, err := pgxpool.New(ctx, os.Getenv("DATABASE_URL"))
if err != nil {
    log.Fatalf("connect: %v", err)
}
defer pool.Close()

var name string
err = pool.QueryRow(ctx,
    "SELECT name FROM users WHERE id = $1", userID,
).Scan(&name)
```

■ Postgres uses $1, $2... placeholders, not ? like MySQL/SQLite. Never build queries with fmt.Sprintf — that's the direct road to SQL injection.

sqlx — StructScan

If you use database/sql directly (rather than pure pgx), sqlx removes the field-by-field Scan() boilerplate:

```
type User struct {
    ID    string `db:"id"`
    Name  string `db:"name"`
    Email string `db:"email"`
}

var u User
```

```
err := db.GetContext(ctx, &u, "SELECT * FROM users WHERE id = $1", id)
```

goose Migrations

migrations/00001_create_urls.sql

```
-- +goose Up
CREATE TABLE urls (
    id         TEXT PRIMARY KEY,
    long_url   TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    hits       INTEGER NOT NULL DEFAULT 0
);
-- +goose Down
DROP TABLE urls;
```

run it

```
goose -dir migrations postgres "$DATABASE_URL" up
```

■ Migrations are the DB equivalent of git commits — every schema change is a new file, never an edit to an old one, so the history stays replayable on a fresh DB.

SQL Aggregation

```
SELECT date_trunc('day', created_at) AS day, COUNT(*) AS hits
FROM url_visits
WHERE url_id = $1
GROUP BY day
ORDER BY day DESC;
```

04 Stage 3 — Auth

The goal is a correct mental model you can debug yourself — not memorizing a library's API surface. Do this once by hand; every future auth integration (OAuth, session-based, whatever a job throws at you) will make more sense because of it.

bcrypt Password Hashing

```
hash, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost)
// store hash, discard password immediately

err = bcrypt.CompareHashAndPassword(storedHash, []byte(attempt))
// nil error == match
```

■ bcrypt is deliberately slow — that's the point. SHA-256 is fast, which makes it terrible for passwords: fast hashing is exactly what makes brute-forcing cheap. Never roll your own here.

JWT Structure

A JWT is three base64url segments joined by dots: `header.payload.signature`. The signature proves the token wasn't tampered with — it does **not** encrypt the payload. Anyone can base64-decode the payload and read it.

golang-jwt/jwt v5

```
type Claims struct {
    UserID string `json:"sub"`
    jwt.RegisteredClaims
}

claims := Claims{
    UserID: user.ID,
    RegisteredClaims: jwt.RegisteredClaims{
        ExpiresAt: jwt.NewNumericDate(time.Now().Add(15 * time.Minute)),
        IssuedAt:  jwt.NewNumericDate(time.Now()),
    },
}
token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
signed, err := token.SignedString([]byte(secret))
```

■ Never put a password, secret, or anything sensitive in JWT claims. Treat the payload as public — assume it's being read on jwt.io by whoever holds the token.

Access vs Refresh Token Lifecycle

Token	Role
Access token	Short-lived (5–15 min). Sent on every request, verified statelessly (no DB hit). Short life limits damage if leaked.
Refresh token	Long-lived (days–weeks). Used only to mint a new access token. Stored server-side (DB) so it can be revoked.

Token	Role
Rotation	Each refresh call issues a *new* refresh token and invalidates the old one — limits replay if a refresh token is stolen.

Auth Middleware

```
func RequireAuth(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        tokenStr := strings.TrimPrefix(r.Header.Get("Authorization"), "Bearer ")
        claims, err := parseAndVerify(tokenStr)
        if err != nil {
            writeError(w, http.StatusUnauthorized, "invalid_token", "authentication requir
ed")
            return
        }
        ctx := context.WithValue(r.Context(), userIDKey, claims.UserID)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

Revocation via DB Blocklist

Stateless access tokens can't be revoked directly — you check the refresh token (which *is* stateful) instead. On logout, delete or flag the refresh token row; the access token simply expires naturally within minutes.

05 Drills

Twelve targeted exercises, one concept each, spanning all three stages. Do these before or alongside the projects — each is small enough to finish in 30–60 minutes.

D-01 Manual Path Params

Stage 1

Register GET /users/{id} and POST /users on one ServeMux. Return 404 with the JSON error envelope for unknown IDs.

Concepts: ServeMux patterns, PathValue

■ **Gotcha:** Pattern precedence is automatic — you don't need to check method inside the handler when the pattern already includes it.

D-02 Middleware Stack

Stage 1

Write Logging, Recovery, and a request-ID middleware. Chain all three. Verify a deliberate panic() in a handler is caught by Recovery and returns 500, not a crashed server.

Concepts: Middleware pattern, panic recovery

■ **Gotcha:** Recovery middleware must be the outermost wrapper — if it's inside another middleware that panics, it won't catch it.

D-03 Graceful Shutdown Drill

Stage 1

Add a handler that sleeps 5 seconds. Send SIGINT mid-request. Confirm the request completes before the process exits, and a second request after signal is rejected.

Concepts: context.WithTimeout, os/signal

■ **Gotcha:** If your shutdown timeout is shorter than the in-flight request, Shutdown() gives up and the request is dropped anyway — size the timeout deliberately.

D-04 Table-Driven Handler Test

Stage 1

Write a table-driven test for one handler with 4+ cases (valid, missing field, bad JSON, not found). Use httptest.NewRecorder — no real server needed.

Concepts: table-driven tests, httptest

■ **Gotcha:** Each test case name matters — go test -run will fail loudly if two cases share a name.

D-05 Connection Pool Behavior

Stage 2

Set pool max connections to 2. Fire 10 concurrent queries with a deliberate 1s sleep in each (pg_sleep(1)). Time it — observe queuing, not failure.

Concepts: pgxpool config, connection pooling

■ **Gotcha:** A pool that's too small doesn't error — it just serializes silently, which looks like a mystery slowdown in production if you don't know to check pool size.

D-06 JWT Tamper Test

Stage 3

Issue a token, manually flip one character in the payload segment, and try to verify it. Confirm verification fails. Then decode (don't verify) the original payload and print it.

Concepts: JWT signature verification

> ■ **Gotcha:** This is the exercise that makes "the signature doesn't encrypt" click — decoding without a secret is trivial; verifying without the secret is not.

D-07 Transaction Rollback

Stage 2

Write a two-step transfer function inside a single tx (debit one row, credit another). Force the second step to fail deliberately and confirm the first step's change never persists.

Concepts: sql.Tx, defer tx.Rollback()

> ■ **Gotcha:** defer tx.Rollback() right after BeginTx is always safe — it's a no-op after a successful Commit, so you don't need an if/else to decide whether to call it.

D-08 N+1 Query Hunt

Stage 2

Write a handler that fetches N users then loops to fetch each user's posts individually. Log query count. Rewrite as a single query with a JOIN or IN (...) and compare counts.

Concepts: JOIN vs per-row queries

> ■ **Gotcha:** The per-row version still "works" and passes tests at small N — the bug only shows up as latency once the row count grows, which is exactly why it's easy to ship by accident.

D-09 Error Envelope Audit

Stage 1

Write one test that hits every registered route with a deliberately bad request and asserts the response body always matches the Envelope{Data, Error} shape, no exceptions.

Concepts: JSON envelope contract, table-driven tests

> ■ **Gotcha:** It's tempting to let one "special" handler (e.g. a health check) skip the envelope — decide that up front instead of discovering the inconsistency later from a confused client.

D-10 Refresh Rotation End-to-End

Stage 3

Script the full cycle: login, capture refresh token, call /refresh, then try reusing the original refresh token. Confirm the reused token is rejected.

Concepts: Refresh token rotation, reuse detection

> ■ **Gotcha:** If reuse of an old refresh token silently succeeds, rotation isn't actually protecting anything — this drill is the test that proves rotation is wired correctly, not just present.

D-11 Context Cancellation Through the Repo Layer

Stage 2

Run a query with pg_sleep(5) via a repo method. Cancel the request context after 1 second and confirm the query is actually aborted on the DB side, not just abandoned client-side.

Concepts: context propagation into pgx calls

> ■ **Gotcha:** A context that's checked in the handler but not threaded into the repo's query call gives you a client that gives up while the DB keeps working — wasted load with no visible symptom.

D-12 bcrypt Cost Benchmark

Stage 3

Benchmark bcrypt.GenerateFromPassword at cost factors 10 through 14. Plot time per hash and pick a cost value for this service with a one-line justification.

Concepts: bcrypt cost factor, testing.B benchmarks

■ **Gotcha:** Cost is exponential, not linear — going from 10 to 14 isn't a 40% slowdown, it's roughly 16x. There's a real login-latency budget to weigh against brute-force resistance.

06 Project 1 — REST API, No Framework

Difficulty ■■■ Timeline **4–5 days** Builds on **Phase 2 net/http server**

Pick a resource you haven't modeled before — tasks, notes, or bookmarks all work. New patterns layer onto whatever routing you already have from Phase 2; this is not a rebuild of that server, it's the same muscles applied to a fresh resource plus everything Stage 1 adds.

Phase 1: Routing skeleton

- Define the resource's CRUD routes on ServeMux using Go 1.22+ method+path patterns
- Stub every handler to return 501 Not Implemented so the shape exists before the logic

Phase 2: Envelope + validation

- Build the Envelope{Data, Error} response helper and use it in every handler, no exceptions
- Add request validation with clear 400s — reject before touching any business logic

Phase 3: Middleware chain

- Write Logging, Recovery, and a Content-Type-enforcement middleware
- Chain them with a Chain() helper — verify order matters by deliberately swapping two

Phase 4: Graceful shutdown

- Wire SIGINT/SIGTERM to context.WithTimeout + srv.Shutdown
- Test it manually: hit a slow endpoint, send the signal, confirm it finishes cleanly

Phase 5: Table-driven tests

- Cover each handler with a table-driven test using httptest.NewRecorder
- Include at least one failure-path case per handler, not just the happy path

You will learn: ServeMux method+path routing, middleware chain composition, JSON response envelopes, structured error handling, graceful shutdown, table-driven HTTP handler tests.

07 Project 2 — URL Shortener + PostgreSQL

Difficulty ■■■■ Timeline **5–6 days** Driver **pgx / pgxpool**

Deceptively small surface area (2–3 real endpoints) hiding a full persistence stack: schema design, migrations, unique ID collision handling, and click analytics.

Phase 1: Schema + migrations

- Design the urls and url_visits tables; write the goose up/down migration pair
- Run the migration against local Postgres, confirm both directions work

Phase 2: Create + redirect

- POST /shorten generates a short ID (random base62, collision-checked) and stores the long URL
- GET /{shortID} looks up and issues http.Redirect — log a row to url_visits on every hit

Phase 3: pgxpool wiring

- Replace any in-memory map with pgxpool.Pool, passed via closure DI like your Phase 2 handlers
- Every query call takes ctx from the incoming request — no context.Background() in handlers

Phase 4: Analytics endpoint

- GET /{shortID}/stats returns total hits + a per-day breakdown via GROUP BY
- Return 404 (not empty stats) for a short ID that was never created

Phase 5: Robustness

- Handle the unique-constraint race: two requests generating the same short ID concurrently
- Add a request timeout via context so a stalled DB doesn't hang the handler forever

You will learn: goose migrations, pgxpool connection pooling, context-scoped queries, http.Redirect, unique constraint / conflict handling, SQL aggregation, environment-based DB config.

08 Project 3 — Auth Service

Difficulty ■■■■■ Timeline **6–7 days** Why last **Needs Stage 1 + 2 fully solid**

The highest-stakes topic in this phase — a shaky mental model here costs the most later. Build it once, by hand, so you can debug any auth system afterward instead of just trusting a library.

Phase 1: Signup + login

• POST /signup hashes password with bcrypt before insert, returns 201 with the created user (no password field)
• POST /login verifies via bcrypt.CompareHashAndPassword, returns 401 on any mismatch — same message for wrong email or wrong password

Phase 2: Token issuance

• On successful login, issue a short-lived access JWT (15 min) and a long-lived refresh token (7 days, stored in DB)
• Design the Claims struct — sub, exp, iat only. Nothing sensitive in the payload

Phase 3: Auth middleware

• Write RequireAuth middleware: parse Bearer token, verify signature + expiry, inject user ID into context
• Protect a sample route with it; confirm 401 on missing/invalid/expired tokens

Phase 4: Refresh + rotation

• POST /refresh validates the refresh token against the DB, issues a new access + refresh pair
• Invalidate (delete or mark used) the old refresh token row on every successful rotation

Phase 5: Revocation

• POST /logout deletes the refresh token row — access token still works until natural expiry, by design
• Write a test that confirms a used/revoked refresh token is rejected on reuse

You will learn: bcrypt hashing, golang-jwt/jwt v5 sign/verify, access vs refresh token design, auth middleware with context injection, DB-backed refresh token rotation and revocation.

09 Patterns Reference

Canonical patterns for this phase. You will reuse all of them past Phase 3.

Middleware Chaining

Wrap outward-in, execute outward-in on the way in, inward-out on the way out — same mental model as an onion or a call stack.

```
Chain(handler, Logging, Recovery, Auth)
// request flow:  Logging -> Recovery -> Auth -> handler
// response flow: handler -> Auth -> Recovery -> Logging
```

Repository Pattern

Isolate SQL behind an interface so handlers never import database/sql or pgx directly — this is what makes handler tests possible without a real database.

```
type UserRepo interface {
    Create(ctx context.Context, u User) error
    GetByEmail(ctx context.Context, email string) (User, error)
}

type pgUserRepo struct{ pool *pgxpool.Pool }
func (r *pgUserRepo) Create(ctx context.Context, u User) error { ... }
// handlers depend on UserRepo, tests use an in-memory fake
```

Context-Aware Queries

Every repo method takes ctx as the first argument and passes it straight into the pgx/database call — this is what makes a client disconnect actually cancel a slow query instead of leaving it running.

Claims Design

Keep JWT claims minimal: subject (user ID), issued-at, expiry. Resist the urge to cram roles, permissions, or profile data in — that data goes stale the moment it's revoked or changed, because the token itself can't be edited after issuance.

10 Common Mistakes & Gotchas

SQL injection via Sprintf CRITICAL	Building queries with fmt.Sprintf("...WHERE id = %s", id) is injectable. Always use parameterized placeholders ($1, $2...) and let the driver handle escaping.
Forgetting rows.Close() CONNECTION LEAK	Every *sql.Rows must be closed, even if you return early on an error mid-loop. defer rows.Close() immediately after the query call, same discipline as resp.Body.Close() in Phase 2.
N+1 queries PERFORMANCE	Looping over results and issuing one query per row (e.g. fetching each user's posts individually) silently kills performance at scale. Use a JOIN or a single IN (...) query.
Committing a Tx and forgetting Rollback on error paths DATA CORRUPTION	defer tx.Rollback() immediately after BeginTx — a Rollback after a successful Commit is a no-op, so this pattern is always safe and covers every early-return error path.
Storing context in a struct STALE CONTEXT	Same rule as Phase 2 — pass ctx explicitly through every function signature down to the DB call. Never store it on a repo or service struct.
Symmetric-key JWT secret reused across environments SECURITY	Using the same signing secret in dev and prod means a leaked dev token or secret compromises prod. Separate secrets per environment, loaded from env vars — never hardcoded.
Treating JWT payload as private SECURITY	The payload is base64, not encrypted — readable by anyone with the token. Never put a password, internal ID you don't want exposed, or PII beyond what's already public in it.
Access token used for long-lived sessions SECURITY	If an access token doesn't expire quickly, a leaked token is a leaked session indefinitely. Keep access tokens short (5–15 min) and lean entirely on refresh rotation for longevity.
Shutdown timeout shorter than slowest handler DROPPED REQUESTS	srv.Shutdown(ctx) gives up and force-closes connections once ctx expires — size the timeout against your slowest realistic request, not an arbitrary round number.
Ignoring sql.ErrNoRows WRONG ERROR SURFACED	QueryRow's Scan returns sql.ErrNoRows for zero matches, not a generic error. Check for it explicitly with errors.Is and map it to 404 — otherwise it looks like a 500 to the client.

11 Resources & What's Next

Canonical Resources for Phase 3

Alex Edwards — Let's Go & Let's Go Further Written reference · alexedwards.net	The definitive practical stdlib net/http guide — covers routing, middleware, and structuring a real service without a framework. Required for Stage 1.
Go Blog — database/sql Tutorial 30 min read · go.dev/doc/database/	Official primer on connection pooling, prepared statements, and the sql.DB/Rows/Row model. Read before Stage 2.
jackc/pgx Documentation Reference · github.com/jackc/pgx	Canonical source for pgxpool configuration and native pgx query patterns.
goose Reference · github.com/pressly/goose	Migration tool docs — up/down conventions, versioning, embedding migrations in a binary.
jwt.io Debugger Interactive · jwt.io	Decode any JWT and see the header/payload/signature split visually — use this during the JWT Tamper Test drill.
golang-jwt/jwt v5 Reference · github.com/golang-jwt/jwt	The library used for signing and verification in Stage 3 — v5 API differs meaningfully from v4, confirm version.
100 Go Mistakes — Ch. 12 & 13 Reference · Teiva Harsanyi	Chapters on REST APIs and security-adjacent mistakes — read alongside the projects.

Phase 4 Preview — What's Coming

Framework retrofit — Rebuild Stage 1's routing/middleware with Chi — diff the line count and API ergonomics
Testing depth — testify, mocking repositories, integration tests against a real test DB (testcontainers)
Observability — Structured logging with log/slog, request tracing, basic metrics
gRPC — Protobuf service definitions, streaming, interceptors
Deployment — Dockerizing the service, environment config, basic CI

■ **Phase 3 completion checkpoint:** You have built a stdlib REST API with middleware and graceful shutdown, a PostgreSQL-backed URL shortener with migrations, and a full auth service with JWT + refresh rotation. You can explain the access/refresh split from memory and debug a broken JWT by hand. You are ready for Phase 4.